CineMatch - Movie Recommendation System

Complete Project Documentation

1. Project Overview

CineMatch is a modern, full-stack Movie Recommendation System designed to help users discover their next favorite film. It bridges the gap between massive global blockbusters (like *Interstellar* and *Inception*) and massive regional hits (like *Baahubali* and *Pushpa*).

Instead of relying on basic keyword searches, CineMatch uses Machine Learning (TF-IDF and Cosine Similarity) to analyze the core thematic DNA (genres) of over 100 curated, top-tier movies. When a user inputs a film they love, the math-driven backend instantaneously returns 6 highly-correlated recommendations. It features a highly interactive 3D frontend, dynamic search, multi-language filtering (English & Telugu), and automatic fallback data retrieval (IMDB via OMDB API & Wikipedia API) for rich movie details.

2. Technology Stack & Required Files

Everything required to run this project is contained within the root folder (`D:\Movie-Recommendation-System`).

Frontend (Located in `/frontend` folder):

- `package.json`: Node.js dependencies (React, Vite, TailwindCSS, Framer Motion, Three.js).
- `src/App.jsx`: Main React application logic.
- `src/components/Scene.jsx`: 3D interactive background.
- `src/components/MovieModal.jsx`: Cinematic popup for IMDB data.

Backend (Located in root folder):

- `app.py`: The core FastAPI Python server logic.
- `requirements.txt`: Python dependencies (`fastapi`, `uvicorn`, `pandas`, `scikit-learn`, `numpy`, `scipy`).
- `movies.db`: The SQLite Database containing all movie titles and genres.
- `render.yaml`: Configuration for deploying the backend to the cloud.
- `generate_movies.py`: Script used to inject top-tier Telugu and English movies into `movies.db`.

3. How Recommendations Work (The Algorithm)

The recommendation engine does not rely on hard-coded rules. Instead, it uses Natural Language Processing (NLP) and vector mathematics to find similarities:

1. **Feature Extraction**: Every movie in the database has a string of metadata associated with it (its `genre`, e.g., "Action Adventure Sci-Fi").
2. **TF-IDF Vectorization (Term Frequency-Inverse Document Frequency)**: The `scikit-learn` library converts these text tags into mathematical numbers. It gives a

higher "weight" to unique or rare tags and lower weight to very common ones. This prevents generic tags from dominating the results.
3. **Cosine Similarity**: The algorithm plots these number vectors in a multi-dimensional space. It then measures the angle between the vector of the searched movie and every other movie in the database. A smaller angle results in a score closer to 1.0 (highly similar), meaning the thematic DNA of the two movies match.
4. **Ranking**: The backend ranks the entire database based on this similarity score and returns the top 6 closest matches to the frontend.

4. Backend Architecture (Python / FastAPI)

The backend serves as the "brain" of the application, handling data processing, machine learning logic, and serving endpoints to the frontend.

- `app.py` loads the SQLite database on startup, vectorizes the movie genres using `scikit-learn`'s `TfidfVectorizer`, and computes a Cosine Similarity matrix to mathematically determine how related movies are to one another.
- **Endpoints**:
  - `/movies` (GET): Returns a list of all movies, with optional language filtering, used for the frontend search autocomplete.
  - `/recommend` (GET): Accepts a movie title, finds its index in the similarity matrix, and returns the top 6 highest-scoring recommendations.

5. Frontend Architecture (React / Vite)

- The user interacts with the UI in `App.jsx`, which communicates with the Python backend to calculate math-based recommendations.
- When a user clicks "View Details", `MovieModal.jsx` queries the **IMDB database** (using the open OMDB wrapper) to fetch the official poster, IMDB rating out of 10, plot summary, and cast.
- If IMDB fails to find the movie, it automatically searches **Wikipedia** and parses its REST API to extract a summary and thumbnail image.

6. The Movie Database (`movies.db`)

The database has been carefully curated to include iconic, high-rating films across two distinct languages to prove the system's filtering and similarity capabilities:

- **English Cinema**: Features massive sci-fi and action epics like *Avatar*, *The Matrix*, *Dune*, and *Oppenheimer*, alongside emotional dramas like *The Shawshank Redemption* and *Forrest Gump*.
- **Telugu Cinema**: Features global phenomena like *RRR* and *Kalki 2898 AD*, alongside beloved regional classics like *Arjun Reddy*, *Jersey*, *Ala Vaikunthapurramuloo*, and *Eega*.

7. Deployment URLs

- **Live Frontend URL**: https://movierecomended.vercel.app
- **Live Backend URL**: https://movie-recommendation-system-d6qr.onrender.com

When users interact with the Vercel site, API calls are securely routed to the Render backend to calculate recommendations, which are then enriched with IMDB/Wikipedia data directly in the browser.